The Business Rule Completeness Conjecture (BRCC) and Its Proof Sketch

Rethinking Conceptual Models – Beyond Syntax

EJ Alexandra
start@anabstractlevel.com
SSoT.me & EffortlessAPI.com
@eejai42
424-242-5558
January 2025

Because once you see it, you cant unsee it!

Contents

1 Concrete Applications and Implications **2**
- 1.1 Minimizing Ripple Effects and Model Co-Evolution . . . 2
- 1.2 Reusable Transformations and Tooling . . . 2
- 1.3 Falsifiability in Practice . . . 3

2 Introduction **4**
- 2.1 The Business Rule Completeness Conjecture (BRCC) . . . 4
- 2.2 Motivation . . . 4
- 2.3 Related Work . . . 4

3 Statement of the Conjecture **5**
- 3.1 Falsifiability . . . 5

4 Proof Sketch of Turing-Completeness **5**
- 4.1 Business Rules as Computable Functions . . . 5
- 4.2 Representing a Universal Model in (S, D, L, A, F) . . . 5
- 4.3 Time and Event-Driven Logic . . . 6

5 Discussion and Practical Caveats **6**

6 Conclusion **6**

Abstract

We introduce and formally motivate the *Business Rule Completeness Conjecture (BRCC)*. It posits that all declarative semantics for any conceptual model—whether an app, workflow, or other domain—can be *unambiguously captured* using only five core primitives: **Data** in an **ACID**-compliant **Schema** that supports **Lookups**, **Rollups (Aggregations)**, and **Calculated Fields**, treating time as just another dimension. We present a theoretical proof sketch, inspired by Turing-completeness arguments, showing that these five primitives are sufficient to encode any finite, computable business rule. Thus, no external syntax or formalism is fundamentally necessary to express the rulebook (the declarative *what*), apart from an imperative engine that executes these definitions (the *how*).

1 Concrete Applications and Implications

1.1 Minimizing Ripple Effects and Model Co-Evolution

A major source of complexity in modern software or Model-Driven Engineering (MDE) arises when the conceptual schema (metamodel, M2) changes, causing ripple effects through the lower-level models (M1) and code or data structures (M0). Under BRCC, however, the metamodel and model are fundamentally locked together as two sides of the same coin in a single database structure. When the M2 level changes, the M1 level updates atomically and consistently, thanks to the ACID properties. This mitigates the typical model repair process, in which partial updates or version mismatches cause the system to drift out of sync.

Model Repair and ACID. Because BRCC systems reside in ACID-compliant data stores, the model and metamodel remain valid as a single transaction. There is no partial state in which the new M2 is defined but the M1 hasnt caught up. Both are stored consistently in the same environment, so no separate transformer steps are needed to keep them aligned. By construction, each instance record and its schema maintain coherence.

1.2 Reusable Transformations and Tooling

BRCC is domain-agnostic, so general-purpose transformations become possible. Because the entire environment (including metamodel definitions and instance data) is stored in a single place, one can build or reuse transformations without domain-specific code. For instance:

- **ssotme:// Protocol and CLI**: An open-source mechanism akin to *git for business rules* that allows version control of all domain definitions (metamodel) and instance data (model) in sync.
- **Airtable-to-Django, Baserow-to-DotNet, etc.**: Over 300 free or open-source tools demonstrate how changes at the conceptual (S, D, L, A, F) level automatically flow into domain-specific environments.

Because these are built on the same fundamental primitives, updates do not require rewriting specialized code or one-off transformations. The ripple effect is replaced by a cohesive environment in which metamodel changes propagate seamlessly.

1.3 Falsifiability in Practice

BRCC is explicitly *falsifiable*. To invalidate it, one need only produce a business rule that can be:

1. Expressed in natural language or a standard programming environment,
2. Yet *cannot* be represented using Schema, Data, Lookups, Aggregations, and Calculated Fields under ACID.

Decades of experience (and attempts at counterexamples) have thus far not yielded any such rule. This practice-based evidence aligns with the formal argument in Section 4: once the system can simulate universal computation, it suffices for capturing all finite, computable business rules. In short, BRCC remains standing, practically and theoretically, until someone finds a concrete counterexample.

2 Introduction

2.1 The Business Rule Completeness Conjecture (BRCC)

The conjecture states:

> *"The declarative semantics for any conceptual model (app or otherwise) can be unambiguously captured with just 5 simple primitives: data, in an ACID-compliant schema that supports lookups, rollups, and calculated fields. Time is treated as just another dimension, and the core difference is between the rulebook (the declarative **what**) versus the runtime engine (the imperative **how**)."*

In other words, if you have:

1. **Schema (S)**: A structure of tables/fields in which your data resides.
2. **Data (D)**: The actual records populating these tables (the facts).
3. **Lookups (L)**: References (foreign keys) that connect rows and form relationships.
4. **Aggregations (A)** (or rollups): Summaries of data over groups (count, sum, etc.).
5. **Calculated Fields (F)**: Formulas or expressions that derive new values from existing data.

plus ACID compliance (atomic, consistent, isolated, and durable transactions), then any business rule—no matter how sophisticated—can be defined within that environment, without resorting to external syntaxes or specialized logic languages.

2.2 Motivation

This conjecture challenges the notion that complex business logic necessarily requires specialized rule engines or domain-specific languages. Instead, it hypothesizes that the fundamental building blocks inherent to many modern databases (e.g., reference/lookup fields, formula fields, rollups) suffice to capture *all* finite, computable rules. From a computer science perspective, business rules are ultimately just functions or predicates over finite data, which in turn are governed by standard computability theory.

2.3 Related Work

The conceptual roots trace back to Turing [1] and Church [2], who established that any effectively computable function can be simulated by a universal model of computation. In the database realm, Codds relational model [3] introduced the concept of schemas, data, and relational integrity constraints, which form the foundation for modern ACID-compliant systems.

3 Statement of the Conjecture

BRCC:

> *For any finite, computable business rule r, there exists an encoding of r as a combination of (S, D, L, A, F) in a single ACID-compliant datastore, such that the rule is captured unambiguously without external syntactic artifacts.*

3.1 Falsifiability

To falsify BRCC, one must exhibit a finite, computable rule **r** that cannot be realized purely within the five primitives. Because of the bridging to Turing-completeness, this would require **r** to be a *computable* function that somehow defies standard universal constructions—an unlikely scenario under current computability theory.

4 Proof Sketch of Turing-Completeness

4.1 Business Rules as Computable Functions

A business rule is any finite statement about how system outputs or constraints should be determined from system inputs. Computability theory tells us such rules correspond to partial or total recursive functions over finite data structures.

4.2 Representing a Universal Model in (S, D, L, A, F)

Let us treat (S, D, L, A, F) within an ACID datastore as follows:

1. **Schema (S)** and **Data (D)** store the tape or state of a Turing machine (or an equivalent universal computational model).
2. **Lookups (L)** connect rows that represent consecutive tape cells or transitions.
3. **Aggregations (A)** summarize and group data, enabling us to handle repeated structure or count transitions.
4. **Calculated Fields (F)** apply formula-based operations to combine data from multiple lookups, effectively executing the universal transition function (e.g., If current state is Q1 and tape symbol is 0, write 1 and move to Q2).

By appropriately structuring these schema objects and formulas, we can reproduce each step of a universal computation. ACID ensures that intermediate steps remain coherent (atomic, consistent) and that references are not broken (isolation, durability).

4.3 Time and Event-Driven Logic

Many business rules rely on the last 30 days or status changed from X to Y. Under the conjecture, these are addressed by storing timestamps or events as rows in Data (S+D), referencing them via Lookups, filtering or grouping with Aggregations, and deriving time-based logic in Calculated Fields. Thus, no additional temporal syntax is needed; time is just another dimension in the data model.

5 Discussion and Practical Caveats

Unbounded Formulas. The Turing-completeness argument often requires potentially unbounded formula complexity and iterative referencing. Real systems (e.g., many no-code/low-code platforms) may cap recursion depth or formula length, thereby restricting pure completeness.

Execution vs. Declaration. BRCC emphasizes the *declarative* dimension—what the rule *means*. The *imperative* dimension of *how* the system executes or optimizes that logic is a separate layer (the runtime engine).

Formalization. A full formal proof would specify the exact syntax and semantics for (S, D, L, A, F) and walk through how to simulate each step of a known universal computing mechanism. The above sections provide a comprehensive sketch sufficient for conceptual acceptance.

6 Conclusion

We have stated and motivated the **Business Rule Completeness Conjecture (BRCC)**, asserting that five core primitives in an ACID datastore—Schema, Data, Lookups, Aggregations, and Calculated Fields—can encode *any* finite, computable business rule. A Turing-completeness style argument underscores why no additional syntactic formalism is strictly necessary to capture the entire rulebook of a system. By storing everything in a single consistent schema, we ensure that the logic remains unambiguous and self-contained.

Hence, *BRCC* stands as a theoretically well-founded claim, awaiting a counterexample—which standard computability theory indicates should not exist—to falsify it.

Q.E.D.

References

[1] A. M. Turing, "On Computable Numbers, with an Application to the Entscheidungsproblem," *Proc. London Math. Soc.*, vol. 2, no. 42, pp. 230–265, 1936.

[2] A. Church, "An Unsolvable Problem of Elementary Number Theory," *American Journal of Mathematics*, vol. 58, no. 2, pp. 345–363, 1936.

[3] E. F. Codd, "A Relational Model of Data for Large Shared Data Banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.